´o
ó
ó
ı

ó

Cordero's Polynomial Theorem (1) and Deterministic Generation of Massive Composite Numbers via Binary Quadratic Forms

Ronald Cordero Méndez

October 12, 2026

Abstract

This paper presents a rigorous formulation of Cordero's Polynomial Theorem (1) and its application to the decomposition and deterministic generation of composite numbers using generalized binary quadratic forms of the type $t^2 - tb + pb^2$, associated with Euler primes $p \in \{2, 3, 5, 11, 17, 41\}$. Unlike traditional heuristic or sub-exponential methods, this approach establishes a constructive algebraic framework that allows for the exact parameterization of the structural symmetry of prime factors. As a result, a parallelizable algorithmic methodology capable of generating and factorizing large-scale composite numbers—surpassing 2,500 digits—with strict control over the density and parity of their components is described. This work provides a structural perspective on recursive algebraic compositions.

Keywords: Number theory, binary quadratic forms, polynomial factorization, deterministic algorithms, recursive algebraic structures.

1 Introduction

The search for deterministic, constructive, and structural methods for the representation and factorization of integers via quadratic forms constitutes one of the fundamental pillars of number theory, from the classical contributions of C.F. Gauss to contemporary computational cryptography. Although modern general-purpose algorithms—such as the General Number Field Sieve (GNFS) or the Elliptic Curve Method (ECM)—offer remarkable asymptotic performance, they predominantly operate under probabilistic or blind-search approaches over high-entropy spaces, often overlooking the internal architecture and intrinsic algebraic symmetry of composite numbers.

This paper addresses the problem from a constructive and algebraic perspective, formalizing Cordero's Polynomial Theorem (1). This theorem establishes a direct and deterministic link between the prime factors generated

through binary quadratic forms of the form $t^2 - tb + pb^2$ (associated with Euler primes $p \in \{2, 3, 5, 11, 17, 41\}$) and a high-density resulting composite number N_P. Through the exact parameterization of auxiliary variables n and r, the proposed methodology allows not only for symmetric decomposition, but also for the controlled generation of large-scale composite numbers exceeding thousands of digits.

The main contributions of this work are the following:

- **The rigorous formulation of Cordero's Polynomial Theorem (1):** Analytical demonstration of the algebraic identity governing the composition of symmetric prime factors using binary quadratic forms.
- **A parallelizable algorithmic framework:** Design of deterministic computational procedures suitable for arbitrary-precision arithmetic, capable of processing massive numerical structures with strict control over parity and density.
- **Empirical evidence of scalability:** Demonstration of the practical viability of the method through the successful generation and validation of symmetric composite numbers of more than 2,500 digits in controlled computation times.

2 Preliminaries and Notation

To establish the framework upon which the present analysis is built, we define the fundamental algebraic sets and domains.

Let $t_1, t_2, b_1, b_2 \in \mathbb{Z}$ be integer variables with the non-zero constraints $b_1 \neq 0$ and $b_2 \neq 0$. Let us also consider a prime number p belonging to the classical set of Euler numbers associated with single-class forms:

$$p \in \{2, 3, 5, 11, 17, 41\}$$

We define the generalized binary quadratic form function as:

$$f(t, b) = t^2 - tb + pb^2$$

These forms operate as the primordial building blocks that determine the arithmetic density and structural symmetry of the evaluated components.

3 Cordero's Polynomial Theorem (1) and Proof

Theorem 3.1 (Cordero's Polynomial Theorem (1))**.** *Let $t_1, t_2, b_1, b_2 \in \mathbb{Z}$ such that $b_1 \neq 0$ and $b_2 \neq 0$, and $p \in \{2, 3, 5, 11, 17, 41\}$.*

If the auxiliary integers n and r are defined by the relations:

$$n = pb_1b_2 - t_1t_2$$

$$r = b_1b_2 - t_1b_2 - t_2b_1$$

Then the polynomial expression:

$$N_P = n^2 - nr + pr^2$$

is a composite number that admits an exact and symmetric factorization of the form:

$$N_P = (t_1^2 - t_1b_1 + pb_1^2)(t_2^2 - t_2b_2 + pb_2^2)$$

Proof. Let $N_P = n^2 - nr + pr^2$, with $n = pb_1b_2 - t_1t_2$ and $r = b_1b_2 - t_1b_2 - t_2b_1$, where $p \in \{2, 3, 5, 11, 17, 41\}$.

Expanding the terms algebraically:

$$N_P = (pb_1b_2 - t_1t_2)^2 - (pb_1b_2 - t_1t_2)(b_1b_2 - t_1b_2 - t_2b_1) + p(b_1b_2 - t_1b_2 - t_2b_1)^2$$

$$N_P = p^2b_1^2b_2^2 - 2pb_1b_2t_1t_2 + t_1^2t_2^2 - (b_1b_2 - t_1b_2 - t_2b_1)(pb_1b_2 - t_1t_2) + p(b_1b_2 - t_1b_2 - t_2b_1)^2$$

$$N_P = t_1^2t_2^2 - b_2t_2t_1^2 + pt_1^2b_2^2 - b_1t_1t_2^2 + b_1b_2t_1t_2 - pt_1b_1b_2^2 + pt_1^2t_2^2 - pt_2b_2b_1^2 + p^2b_1^2b_2^2 \quad (*)$$

On the other hand, expanding the product of factors:

$$(t_1^2 - t_1b_1 + pb_1^2)(t_2^2 - t_2b_2 + pb_2^2) = t_1^2t_2^2 - b_2t_2t_1^2 + pt_1^2b_2^2 - b_1t_1t_2^2 + b_1b_2t_1t_2 - pt_1b_1b_2^2 + pt_1^2t_2^2 - pt_2b_2b_1^2 + p^2b_1^2b_2^2 \quad (**)$$

From $(*)$ and $(**)$ we obtain that:

$$N_P = n^2 - nr + pr^2 = (t_1^2 - t_1b_1 + pb_1^2)(t_2^2 - t_2b_2 + pb_2^2)$$

This completes the analytical validity of the identity for all integer domains under the specified Euler primes. ■ □

4 The Constructive Nature of Cordero's Polynomial Theorem (1)

The constructive nature of this theorem makes it possible to dispense with the probabilistic or trial-and-error methods customary in traditional factorization, replacing them with a deterministic algorithmic procedure.

Algorithm Architecture: The generation and decomposition process is structured into the following operational phases:

1. **High-Precision Parameter Initialization:** The integers b_1, b_2, r and d are selected, where $b_1 \neq 0$, $b_2 \neq 0$ and $\gcd(b_1, b_2) = 1$, alongside an Euler prime parameter $p \in \{2, 3, 5, 11, 17, 41\}$.

2. Let $t_1 = b_1k + \alpha$, with $k \in \mathbb{Z}$.

3. Solving for t_2 from the relation $r = b_1b_2 - t_1b_2 - t_2b_1$:

$$t_2 = \frac{b_1b_2 - t_1b_2 - r}{b_1}$$

4. Substituting $t_1 = b_1 k + \alpha$ into the expression for t_2, modular arithmetic dictates that:
$$\alpha \equiv -r b_2^{-1} \pmod{b_1}$$
Let $t_2 = \beta - k b_2$ with $\beta = \frac{b_2 - \alpha b_2 - r}{b_1}$.

5. **Binary-Form Prime Factor Evaluation:** Symmetric components are calculated independently in terms of a single variable k: $F_1 = t_1^2 - t_1 b_1 + p b_1^2 = b_1^2 k^2 + (2\alpha b_1 - b_1^2)k + (\alpha^2 - \alpha b_1 + p b_1^2)$ $F_2 = t_2^2 - t_2 b_2 + p b_2^2 = b_2^2 k^2 - (2\beta b_2 - b_2^2)k + (\beta^2 - \beta b_2 + p b_2^2)$ Let d be the minimum number of digits of F_1 and F_2, establishing the lower bound 10^{d-1}. Solving the quadratic inequalities yields the minimum starting value $k_{\min} = \max(k_1, k_2)$.

6. **Primality Testing:** The Miller-Rabin probabilistic primality test is used to confirm the primality of F_1 and F_2.

5 Computational Implementation and Parallel Generation Algorithm

Below is the corrected and properly indented Python implementation utilizing arbitrary-precision arithmetic and parallel multiprocessing to generate massive composite numbers of thousands of digits efficiently.

```python
import math
import multiprocessing as mp
import random
import secrets
import time
from sympy import isprime

# Intentar usar gmpy2 para aceleracion de enteros gigantes en C
TRY_GMPY2 = True
try:
    import gmpy2
    from gmpy2 import mpz
except ImportError:
    TRY_GMPY2 = False

PRIMOS_PEQUENOS = [
    3, 5, 7, 11, 13, 17, 19, 23, 29, 31,
    37, 41, 43, 47, 53, 59, 61, 67, 71,
    73, 79, 83, 89, 97, 101, 103, 107,
    109, 113, 127, 131, 137, 139, 149,
    151, 157, 163, 167, 173, 179, 181,
    191, 193, 197, 199
]
```

```
WHEEL = 2310
RESIDUOS_VALIDOS = [r for r in range(WHEEL) if math.gcd(r, WHEEL) == 1]
LOG10_2 = math.log10(2)

def digitos(n: int) -> int:
    if n == 0:
        return 1
    return int(n.bit_length() * LOG10_2) + 1

def es_probable_primo_cripto(n: int, k_rounds: int = 40) -> bool:
    if TRY_GMPY2:
        return gmpy2.is_prime(mpz(n), k_rounds)
    if n < 2:
        return False
    if n in (2, 3):
        return True
    if n % 2 == 0:
        return False
    for q in PRIMOS_PEQUENOS:
        if n == q:
            return True
        if n % q == 0:
            return False
    d = n - 1
    s = 0
    while d % 2 == 0:
        d //= 2
        s += 1
    for _ in range(k_rounds):
        a = random.randint(2, n - 2)
        x = pow(a, d, n)
        if x == 1 or x == n - 1:
            continue
        compuesto = True
        for _ in range(s - 1):
            x = (x * x) % n
            if x == n - 1:
                compuesto = False
                break
        if compuesto:
            return False
    return True

def construir_residuos_prohibidos(A1: int, B1: int, C1: int, A2: int, B2: int, C2: int) -> dict:
    prohibidos = {}
    for q in PRIMOS_PEQUENOS:
```

```
        A1q, B1q, C1q = A1 % q, B1 % q, C1 % q
        A2q, B2q, C2q = A2 % q, B2 % q, C2 % q
        malos = set()
        for k in range(q):
            m1 = (A1q * k * k + B1q * k + C1q) % q == 0
            m2 = (A2q * k * k + B2q * k + C2q) % q == 0
            if m1 and m2:
                malos.add(k)
        prohibidos[q] = malos
    return prohibidos

def k_minimo(A: int, B: int, C: int, target: int) -> int:
    D = B * B - 4 * A * (C - target)
    if D < 0:
        return 0
    raiz = math.isqrt(D)
    return (-B + raiz) // (2 * A)

def worker_buscar_primo_opt(worker_id: int, num_workers: int, base_busqueda: int,
                            A1: int, B1: int, C1: int, A2: int, B2: int, C2: int,
                            prohibidos: dict, stop_event: mp.Event,
                            queue_resultado: mp.Queue):
    rng = random.Random(secrets.randbits(64) + worker_id)
    paso_worker = num_workers * WHEEL
    offset_inicial = base_busqueda + (worker_id * WHEEL)
    primos_tuple = tuple(PRIMOS_PEQUENOS)
    while not stop_event.is_set():
        r_wheel = rng.choice(RESIDUOS_VALIDOS)
        salto = rng.randint(0, 65535)
        K = offset_inicial + (salto * paso_worker) + r_wheel

        cribado = False
        for q in primos_tuple:
            if (K % q) in prohibidos[q]:
                cribado = True
                break
        if cribado:
            continue

        KK = K * K
        F1 = A1 * KK + B1 * K + C1
        F2 = A2 * KK + B2 * K + C2
        cand1 = (F1 % 10) in (1, 3, 7, 9)
        cand2 = (F2 % 10) in (1, 3, 7, 9)
```

```
113         if not cand1 and not cand2:
114             continue
115 
116         es_p1 = cand1 and es_probable_primo_cripto(F1,
                k_rounds=2)
117         es_p2 = cand2 and es_probable_primo_cripto(F2,
                k_rounds=2)
118         if not (es_p1 or es_p2):
119             continue
120 
121         if es_p1 and es_probable_primo_cripto(F1,
                k_rounds=40):
122             if not stop_event.is_set():
123                 stop_event.set()
124                 queue_resultado.put(F1)
125             break
126         if es_p2 and es_probable_primo_cripto(F2,
                k_rounds=40):
127             if not stop_event.is_set():
128                 stop_event.set()
129                 queue_resultado.put(F2)
130             break
131 
132 def buscar_un_primo_paralelo(base_busqueda: int, A1: int,
        B1: int, C1: int,
133                              A2: int, B2: int, C2: int,
                                    prohibidos: dict,
                                    etiqueta_fase: str) ->
                                    int:
134     num_cores = mp.cpu_count()
135     stop_event = mp.Event()
136     queue_resultado = mp.Queue()
137     procesos = []
138     for w_id in range(num_cores):
139         p = mp.Process(
140             target=worker_buscar_primo_opt,
141             args=(w_id, num_cores, base_busqueda, A1, B1,
                        C1, A2, B2, C2,
142                   prohibidos, stop_event, queue_resultado
                          )
143         )
144         p.start()
145         procesos.append(p)
146     primo_encontrado = queue_resultado.get()
147     for p in procesos:
148         p.terminate()
149         p.join()
150     return primo_encontrado
```

6 Experimental Results and Applications

6.1 First Application Example

Inputs: $b_1 = 984295832948957425898295942859475729859342789582094589275 1$,
$b_2 = 882938905809485098290485984329058092348095824957923483058209850928345098290587 01$,
$p = 41$, $r = 9345289485982943859783475792438590829458094237572904850928490820 9$,
Min. Digits: 238.

Execution Results: Factor Prime 1: 281 digits found. Factor Prime 2: 281 digits found. Total Digits of N_P: 562 digits. Execution Time: 1.01 seconds.

6.2 Second Application Example

Inputs: $b_1 = 2772772747952895728590289592834758743857289752489578932475982975427587435684$
$b_2 = 8782748798578927457875872843785728592895082094589827895787589342789658917845297589 78$
$p = 41$, $r = 2757278957827857823478574287584785784327527875724857478957487528789738957834 27$
Min. Digits: 803.

Execution Results: Factor Prime 1: 816 digits found. Factor Prime 2: 803 digits found. Total Digits of N_P: 1618 digits. Execution Time: 40.78 seconds.

6.3 Third Application Example

Inputs: $b_1 = 7234857982775723495847328998934832787897895734728578475824365897589728578927$
$b_2 = 7892743897857389457897452878578923582895789275847358947285378753485828578275874829 38$
$p = 17$, $r = 4838959348905892757247574285784327857342875748957892437889257894782545892789 57$
Min. Digits: 1231.

Execution Results: Factor Prime 1: 1231 digits found. Factor Prime 2: 1233 digits found. Total Digits of N_P: 2463 digits. Execution Time: 85.66 seconds.

7 Main Conclusions of the Research Work

1. **Transition Toward a Constructive and Deterministic Paradigm:** Cordero's Polynomial Theorem (1) breaks away from traditional dependence on purely probabilistic, heuristic, or blind-search methods.

2. **Robust Analytical Rigor:** The proof of the polynomial identity provides unbreakable deductive support.

3. **Computational Scalability and Practical Performance:** The parallelized algorithmic implementation proves that the theory is highly functional up to thousands of digits.

4. **New Cryptographic and Theoretical Perspectives:** Opens a fundamental avenue of research for modern cryptanalysis.

References

1. Cox, D.A. (2013). *Primes of the form $x^2 + ny^2$: Fermat, class field theory, and complex multiplication*. John Wiley & Sons.

2. Gauss, C.F. (1801). *Disquisitiones Arithmeticae*. Fleischer, Leipzig.

3. Cohen, H. (1993). *A Course in Computational Algebraic Number Theory*. Graduate Texts in Mathematics, vol. 138, Springer-Verlag.

4. Crandall, R., & Pomerance, C. (2005). *Prime Numbers: A Computational Perspective* (2nd ed.). Springer.

5. Brent, R.P. (1976). An improved Monte Carlo factorization algorithm. *BIT Numerical Mathematics*, 16(2), 143-156.

6. Python Software Foundation. (2026). *Python Language Reference, version 3.x*.

7. Cordero Méndez, R. (2025–2026). *Investigaciones sobre formas cuadráticas binarias, primos de Euler y teoremas de factorización polinomial*. Mathematical development manuscripts and notes.